

**UNIVERSIDAD MESOAMERICANA**

**INGENIERÍA EN SISTEMAS**

**DOCUMENTACIÓN TÉCNICA**

**Compilador MicroC**

|                  |                             |
|------------------|-----------------------------|
| <b>Nombre:</b>   | Jose Angel Moreno Hernández |
| <b>Carnet:</b>   | 202425514                   |
| <b>Curso:</b>    | Autómatas y Lenguajes       |
| <b>Carrera:</b>  | Ingeniería en Sistemas      |
| <b>Proyecto:</b> | Compilador MicroC           |

Guatemala, 2026

# 1. Introducción

**MicroC** es una aplicación de escritorio desarrollada en C# con Avalonia .NET, orientada a simular las primeras fases de un compilador para un lenguaje similar a C. En su estado actual, el sistema implementa la Fase I y Fase II del analizador léxico, permitiendo transformar código fuente en una lista estructurada de tokens.

La documentación técnica describe el funcionamiento interno del proyecto, las clases principales, el árbol de decisión aplicado, los autómatas implementados y los resultados obtenidos durante las pruebas. Esta versión actualiza la documentación anterior, reemplazando la validación estructural básica por un análisis léxico más completo.

## 2. Objetivos

### 2.1 Objetivo general

Desarrollar un analizador léxico funcional para el compilador MicroC, capaz de reconocer lexemas, clasificarlos como tokens y detectar errores léxicos básicos dentro de código fuente similar al lenguaje C.

### 2.2 Objetivos específicos

- Implementar una interfaz gráfica que permita escribir, abrir, guardar y analizar código fuente.
- Generar una lista de tokens con línea, lexema, número de token y tipo.
- Reconocer palabras reservadas, identificadores, números enteros y números reales.
- Identificar comentarios de línea y comentarios de bloque.
- Detectar errores léxicos como números mal formados, símbolos no reconocidos y comentarios incompletos.
- Aplicar conceptos de autómatas finitos y árboles de decisión en la construcción del analizador léxico.

## 3. Marco teórico

Un compilador es un programa que traduce código fuente escrito en un lenguaje de programación hacia otro lenguaje, normalmente código máquina o una representación intermedia. El proceso de compilación suele dividirse en varias etapas, entre ellas: análisis léxico, análisis sintáctico, análisis semántico, generación de código intermedio, optimización y generación de código final.

El análisis léxico es la primera etapa del proceso. Su función es leer el código fuente carácter por carácter para agruparlos en lexemas y clasificarlos como tokens. Para representar esta lógica se utilizan expresiones regulares, autómatas finitos y árboles de decisión.

| Concepto          | Descripción                                                                                                       |
|-------------------|-------------------------------------------------------------------------------------------------------------------|
| Lexema            | Secuencia de caracteres reconocida dentro del código fuente, por ejemplo int, main, 10.50 o //.                   |
| Token             | Categoría asociada a un lexema. Puede representar palabra reservada, identificador, número, símbolo o comentario. |
| Autómata finito   | Modelo de estados usado para reconocer patrones dentro de una cadena.                                             |
| Árbol de decisión | Estructura lógica que decide qué autómata ejecutar según el carácter actual.                                      |

## 4. Tecnologías utilizadas

| Elemento                        | Tecnología               |
|---------------------------------|--------------------------|
| Lenguaje de programación        | C#                       |
| Framework                       | .NET 8                   |
| Interfaz gráfica                | Avalonia UI              |
| Sistema operativo de desarrollo | Linux / Ubuntu           |
| Control de versiones            | Git y GitHub             |
| Tipo de aplicación              | Aplicación de escritorio |

## 5. Arquitectura del proyecto

El proyecto está organizado principalmente en la carpeta src, donde se encuentra el código fuente de la aplicación. Las clases principales se dividen entre la interfaz gráfica y la lógica del analizador léxico.

| Archivo             | Función principal                                                                                        |
|---------------------|----------------------------------------------------------------------------------------------------------|
| Program.cs          | Punto de entrada de la aplicación Avalonia. Inicializa la aplicación y configura la ventana principal.   |
| MainWindow.axaml    | Define la estructura visual de la interfaz gráfica: botones, editor y consola de salida.                 |
| MainWindow.axaml.cs | Contiene los eventos de la interfaz, incluyendo el botón Compilar, Abrir, Guardar, Nuevo, Ayuda y Salir. |
| TokenLexico.cs      | Modelo de datos para representar cada token con línea, lexema, token y tipo.                             |
| UnidadesLexicas.cs  | Contiene diccionarios de símbolos, operadores y palabras reservadas.                                     |
| AnalizadorLexico.cs | Implementa el recorrido del código fuente y los autómatas de reconocimiento léxico.                      |

## 6. Flujo general del análisis léxico

El análisis léxico se ejecuta cuando el usuario presiona el botón Compilar. El texto escrito en el editor se envía a la clase AnalizadorLexico, donde se recorre carácter por carácter hasta llegar al final del archivo.

```
Inicio
  Crear lista de tokens
  Instanciar UnidadesLexicas
  Mientras contador < longitud del archivo:
    Leer caracter actual
    Decidir el tipo de lexema
    Ejecutar el autómata correspondiente
    Agregar token a la lista
  Retornar lista de tokens
```

## 7. Fase I - Analizador léxico inicial

La Fase I se enfocó en generar una lista de tokens básica. En esta etapa el programa ya podía leer el código fuente, separar lexemas, relacionarlos con su número de línea y mostrarlos en la consola de salida.

- Lectura del código fuente carácter por carácter.
- Identificación de lexemas simples.
- Reconocimiento inicial de símbolos.
- Registro de línea, lexema, token y tipo.
- Conteo total de tokens y errores léxicos.

## 8. Fase II - Analizador léxico completo

La Fase II amplió el analizador para completar la lista de tokens solicitada. Se agregó reconocimiento de palabras reservadas, números enteros y reales, comentarios de línea, comentarios de bloque, operadores compuestos y errores léxicos específicos.

### 8.1 Palabras reservadas e identificadores

El analizador utiliza un mismo autómata para formar lexemas que empiezan con letra o guion bajo. Al terminar de formar el lexema, consulta la tabla de palabras reservadas. Si la palabra existe en el diccionario, se clasifica como PalabraReservada; de lo contrario, como Identificador.

Expresión regular para identificadores:  
[A-Za-z\_][A-Za-z0-9\_]\*

| Ejemplo    | Clasificación    | Token |
|------------|------------------|-------|
| int        | PalabraReservada | 17    |
| return     | PalabraReservada | 20    |
| main       | Identificador    | 300   |
| suma_total | Identificador    | 300   |

### 8.2 Números enteros y reales

El autómata de números inicia cuando el carácter actual es un dígito. Si el lexema contiene únicamente dígitos, se acepta como número entero. Si contiene un punto decimal seguido de uno o más dígitos, se acepta como número real. Si el número termina en punto o contiene más de un punto decimal, se genera un error léxico.

Entero: [0-9]+  
Real: [0-9]+\.[0-9]+  
General: [0-9]+(\.[0-9]+)?

| Lexema | Resultado               |
|--------|-------------------------|
| 25     | NumeroEntero, token 301 |
| 10.50  | NumeroReal, token 302   |
| 3.1416 | NumeroReal, token 302   |
| 8.     | ErrorLexico             |

### 8.3 Comentarios de línea y bloque

Cuando el analizador encuentra el carácter /, revisa el siguiente carácter para decidir si se trata de comentario de línea, comentario de bloque o símbolo de división. Los comentarios de línea se leen hasta el salto de línea; los comentarios de bloque se leen hasta encontrar el cierre \*/.

```
Comentario de línea: // seguido de caracteres hasta antes del salto de línea
Comentario de bloque: /* contenido */
```

| Lexema                   | Tipo             | Token |
|--------------------------|------------------|-------|
| // variable inicial      | ComentarioLinea  | 400   |
| /* comentario */         | ComentarioBloque | 401   |
| /* comentario sin cerrar | ErrorLexico      | -1    |

### 8.4 Operadores compuestos

Además de los símbolos simples, el analizador reconoce operadores compuestos de dos caracteres. Para esto evalúa el carácter actual junto con el siguiente carácter.

| Operador | Token | Tipo    |
|----------|-------|---------|
| ++       | 110   | Simbolo |
| --       | 111   | Simbolo |
| ==       | 112   | Simbolo |
| !=       | 113   | Simbolo |
| <=       | 114   | Simbolo |
| >=       | 115   | Simbolo |
| &&       | 116   | Simbolo |
|          | 117   | Simbolo |
| +=       | 118   | Simbolo |
| -=       | 119   | Simbolo |
| *=       | 120   | Simbolo |
| /=       | 121   | Simbolo |

## 9. Autómatas implementados

### 9.1 Autómata de palabras reservadas e identificadores

```
q0 -- letra o _ --> q1
q1 -- letra, dígito o _ --> q1
q1 -- otro caracter --> aceptar lexema
```

```
Al aceptar:
  si lexema está en tabla -> PalabraReservada
  si no está en tabla    -> Identificador
```

### 9.2 Autómata de números enteros y reales

```
q0 -- dígito --> q1
q1 -- dígito --> q1
q1 -- . --> q2
q1 -- otro --> NumeroEntero
q2 -- dígito --> q3
```

```

q2 -- otro --> ErrorLexico
q3 -- dígito --> q3
q3 -- . --> ErrorLexico
q3 -- otro --> NumeroReal

```

### 9.3 Autómata de comentarios

```

q0 -- / --> q1
q1 -- / --> q2      Comentario de línea
q2 -- cualquier caracter excepto salto de línea --> q2
q2 -- salto de línea o fin de archivo --> ComentarioLinea

q1 -- * --> q3      Comentario de bloque
q3 -- cualquier caracter excepto * --> q3
q3 -- * --> q4
q4 -- / --> ComentarioBloque
q4 -- * --> q4
q4 -- otro --> q3

Si termina el archivo sin encontrar */ -> ErrorLexico

```

## 10. Pruebas realizadas

Para verificar el funcionamiento del analizador léxico se realizaron pruebas manuales desde la interfaz gráfica del programa. Cada prueba confirma un grupo específico de tokens.

| Prueba                | Código de entrada                    | Resultado esperado                                           |
|-----------------------|--------------------------------------|--------------------------------------------------------------|
| Palabras reservadas   | int main() { return 0; }             | int y return como PalabraReservada; main como Identificador. |
| Números               | float precio = 10.50; int edad = 25; | 10.50 como NumeroReal y 25 como NumeroEntero.                |
| Comentario de línea   | int x = 10; // variable inicial      | // variable inicial como ComentarioLinea.                    |
| Comentario de bloque  | /* comentario */ int x = 5;          | ComentarioBloque y luego continúa el análisis del código.    |
| Operadores compuestos | if (x >= 10 && x != 20) { x++; }     | >=, &&, != y ++ como símbolos compuestos.                    |
| Errores léxicos       | 10.5.3, 8., /* comentario sin cerrar | Los tres lexemas se clasifican como ErrorLexico.             |

### 10.1 Ejemplo de salida

```

Entrada:
int main() {
    return 0;
}

Salida resumida:
Linea: 1   Lexema: int       Token: 17   Tipo: PalabraReservada
Linea: 1   Lexema: main     Token: 300   Tipo: Identificador
Linea: 2   Lexema: return   Token: 20   Tipo: PalabraReservada

```

Línea: 2    Lexema: 0    Token: 301    Tipo: NumeroEntero  
Total de tokens: 9  
Errores léxicos: 0

## 11. Enlaces del proyecto

| Recurso                                     | Enlace                                                                                                                                          |
|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Repositorio GitHub                          | <a href="https://github.com/angxlhndz-gt/Compilador-MicroC-AngelHernandez">https://github.com/angxlhndz-gt/Compilador-MicroC-AngelHernandez</a> |
| Video 01 - Fase I Árbol de Decisión MicroC  | <a href="https://youtu.be/OJO1N2i6aJI">https://youtu.be/OJO1N2i6aJI</a>                                                                         |
| Video 02 - Fase II Árbol y Autómatas MicroC | <a href="https://youtu.be/vvKoWBUMEAs">https://youtu.be/vvKoWBUMEAs</a>                                                                         |
| Video 03 - Demo Funcional Compilador MicroC | <a href="https://youtu.be/acdE15tPMkc">https://youtu.be/acdE15tPMkc</a>                                                                         |

## 12. Resultados obtenidos

El sistema cumple con la funcionalidad principal solicitada para la Fase I y Fase II del analizador léxico. La aplicación permite analizar código desde una interfaz gráfica, generar tokens y reportar errores léxicos de manera clara.

- Se generó una lista de tokens con línea, lexema, token y tipo.
- Las palabras reservadas ya no se clasifican como identificadores.
- Los números enteros y reales se diferencian correctamente.
- Los comentarios de línea y bloque son reconocidos como tokens independientes.
- Los errores léxicos se detectan y se contabilizan al final de la salida.
- El repositorio se organizó con README, documentación, capturas, videos y archivos fuente.

## 13. Conclusiones

- El desarrollo del Compilador MicroC permitió aplicar de forma práctica los conceptos de autómatas finitos, expresiones regulares y análisis léxico vistos en el curso.
- La Fase I sentó la base del recorrido carácter por carácter y la generación de tokens, mientras que la Fase II completó el reconocimiento de palabras reservadas, números y comentarios.
- El uso de árboles de decisión facilitó la organización del flujo del analizador, permitiendo decidir qué autómata ejecutar según el carácter actual.
- El proyecto deja una base funcional para continuar con fases posteriores de un compilador, como análisis sintáctico y análisis semántico.

## 14. Recomendaciones

- Ampliar la tabla de tokens si se desea soportar más elementos del lenguaje C.
- Implementar una fase sintáctica que valide estructuras como declaraciones, asignaciones y condicionales.
- Agregar pruebas automatizadas para comprobar casos válidos e inválidos del analizador.
- Mantener actualizado el README y la documentación cada vez que se agregue una nueva fase.